



**INSTITUTO TECNOLÓGICO DE COSTA RICA
ESCUELA DE INGENIERÍA EN COMPUTADORES
CURSO: INTRODUCCIÓN A LA PROGRAMACIÓN**

DOCUMENTO TÉCNICO
PROYECTO: DEFENSA Y ASALTO DE BASE

PROFESORES:
JEFF SCHMIDT PERALTA
DIEGO ANDRÉS MORA ROJAS

ESTUDIANTES:
VALERIA AGUILAR CALDERON
ERNESTO MORA MARTÍNEZ

DESCRIPCIÓN GENERAL DEL JUEGO

Defensa y Asalto de Base es un juego de estrategia por turnos para dos jugadores que se ejecuta en una sola ventana de escritorio construida con Tkinter. Un jugador asume el rol de defensor y debe proteger una base central fija; el otro jugador asume el rol de atacante y debe destruir esa base utilizando distintas unidades.

La partida se desarrolla sobre un tablero cuadriculado de 10×10 casillas. La base central ocupa un bloque fijo de 2×2 casillas en el centro del mapa. Cada jugador dispone de dinero para comprar y colocar elementos en el tablero, y el juego avanza por rondas.

Cada ronda se divide en tres fases: primero la fase de construcción del defensor (muros y torres), luego la fase de ataque (colocación de unidades) y por último la fase de combate, que se resuelve turno por turno. El primer jugador que gane 3 rondas gana la partida completa, y su victoria se registra de forma permanente en el perfil del jugador.

Nota: en la pantalla de inicio el juego se presenta con el título visual «Eclipse of Kingdoms», que corresponde a la identidad gráfica de la portada del proyecto «Defensa y Asalto de Base».

EXPLICACIÓN DE LAS REGLAS

➤ **REGISTRO E INICIO DE SESIÓN**

Antes de comenzar una partida, ambos jugadores deben registrarse o iniciar sesión. Cada jugador tiene un nombre de usuario, una contraseña y un historial de victorias separado para el rol de defensor y para el rol de atacante. El sistema valida que los dos usuarios sean distintos y que las contraseñas sean correctas.

➤ **SELECCIÓN DE FACCIÓN**

Tras iniciar sesión, cada jugador elige una facción entre tres disponibles: medieval, futurista y naturaleza. El defensor y el atacante no pueden usar la misma facción; el programa bloquea el inicio de la partida si ambos seleccionan la misma.

➤ **DESARROLLO DE UNA RONDA**

1. El defensor recibe su dinero inicial y coloca muros y torres en el tablero.
2. El defensor termina su fase y comienza la fase de ataque.
3. El atacante recibe su dinero inicial y coloca sus unidades únicamente en los bordes del mapa.
4. Se inicia la fase de combate, que avanza turno a turno mediante el botón «Ejecutar turno».
5. En cada turno, primero atacan las torres y luego se mueven o atacan las unidades.
6. Se determina el ganador de la ronda y se actualiza el marcador.
7. Si ningún jugador ha ganado 3 rondas, comienza una nueva ronda.

➤ **RESOLUCIÓN DEL COMBATE (POR TURNO)**

Cada vez que se ejecuta un turno, la secuencia de acciones es la siguiente:

- Las torres atacan primero. Cada torre busca la unidad enemiga más cercana que esté dentro de su alcance y le aplica daño. Cuando su contador de turnos alcanza su tiempo de recarga (cooldown), activa además su habilidad especial.
 - Se eliminan del tablero las unidades que se quedaron sin vida y el defensor recibe la recompensa correspondiente por cada una.
-

- Las unidades actúan después. Cada unidad revisa sus cuatro casillas adyacentes: si encuentra la base, una torre o un muro, lo ataca; si no hay nada que atacar, avanza una casilla hacia la base. Las unidades congeladas pierden su turno.
- Se eliminan las torres y muros destruidos, y el atacante recibe dinero por cada torre destruida.

➤ CONDICIONES DE VICTORIA

Victoria del atacante en la ronda: logra reducir a 0 la vida de la base central.

Victoria del defensor en la ronda: elimina todas las unidades atacantes sin que la base sea destruida.

Victoria de la partida: el primer jugador que gana 3 rondas gana la partida; su victoria se suma al historial de su perfil según el rol jugado.

EXPLICACIÓN DE LAS TORRES

Las torres son las defensas que coloca el defensor. El juego incluye tres tipos de torres, cada una representada por la clase Torre. Cada torre tiene un nombre, un costo, vida, daño, alcance y un tiempo de recarga que determina cada cuántos turnos se activa su habilidad especial.

Torre	Costo	Vida	Daño	Alcance	Recarga
Torre básica	60	80	20	2	3 turnos
Torre pesada	110	140	35	2	4 turnos
Torre mágica	90	70	15	3	3 turnos

Habilidades especiales de las torres

- Torre básica con disparo doble: realiza un segundo disparo que aplica daño adicional a la misma unidad.
- Torre pesada con golpe reforzado: añade un golpe extra de 20 puntos de daño sobre la unidad objetivo.
- Torre mágica con congelar: congela a una unidad enemiga, que pierde su siguiente turno y no puede moverse ni atacar.

Las tres habilidades están programadas dentro del método habilidad() de la clase Torre, que se ejecuta según el tipo de torre.

EXPLICACIÓN DE LAS UNIDADES

Las unidades son los elementos de ataque que coloca el atacante en los bordes del mapa. El juego incluye tres tipos de unidades, representadas por la clase Unidad. Además de nombre, costo, vida y daño, cada unidad tiene una velocidad (casillas que avanza por turno), un tiempo de recarga para su habilidad y una recompensa: el dinero que recibe el defensor si la elimina.

Unidad	Costo	Vida	Daño	Velocidad	Recarga	Recompensa (Dinero)
Soldado	45	70	20	1	3 turnos	25
Tanque	95	160	30	1	4 turnos	50
Unidad rápida	65	55	15	2	3 turnos	35

Habilidades especiales de las unidades

- Soldado con ataque doble: ataca dos veces durante ese turno en lugar de una.
- Tanque con escudo temporal: activa un escudo que reduce a la mitad el siguiente daño que reciba; el escudo se gasta tras un solo golpe.
- Unidad rápida con aumento de velocidad: avanza una casilla adicional durante ese turno.

Las tres habilidades están programadas dentro del método habilidad() de la clase Unidad y se invocan según el tipo de unidad.

EXPLICACIÓN DE LAS FACCIÓNES

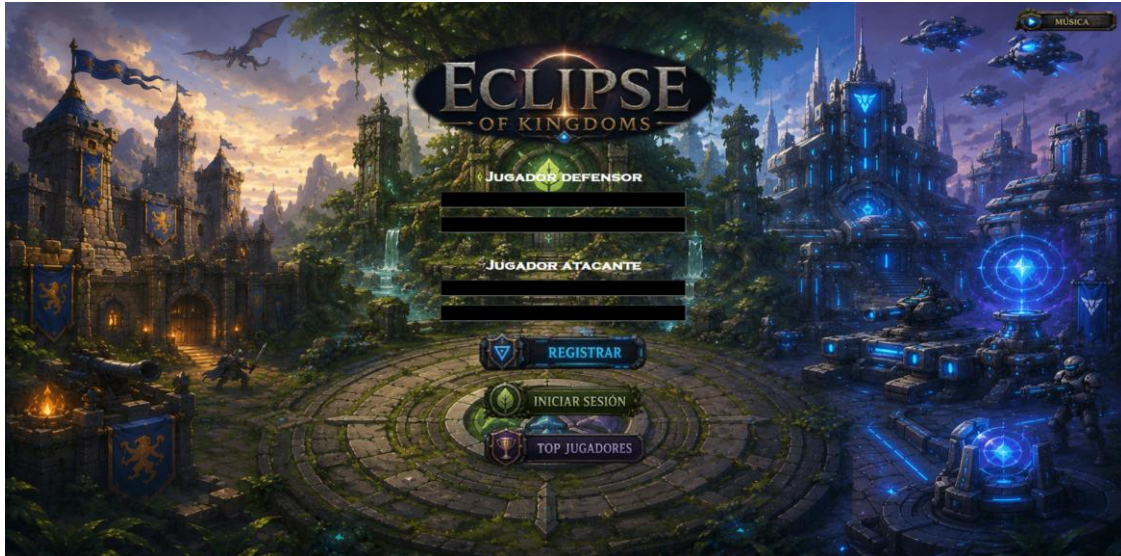
Antes de iniciar la partida, cada jugador elige una de las tres facciones disponibles. Las facciones modifican únicamente el aspecto visual de los elementos del juego (la base central, los muros, las torres y las unidades), sin alterar sus estadísticas, de modo que el juego se mantiene balanceado.

Facción	Temática visual
Medieval	Estética de castillos, piedra y construcciones de época.
Futurista	Estética tecnológica de ciencia ficción.
Naturaleza	Estética orgánica basada en elementos naturales.

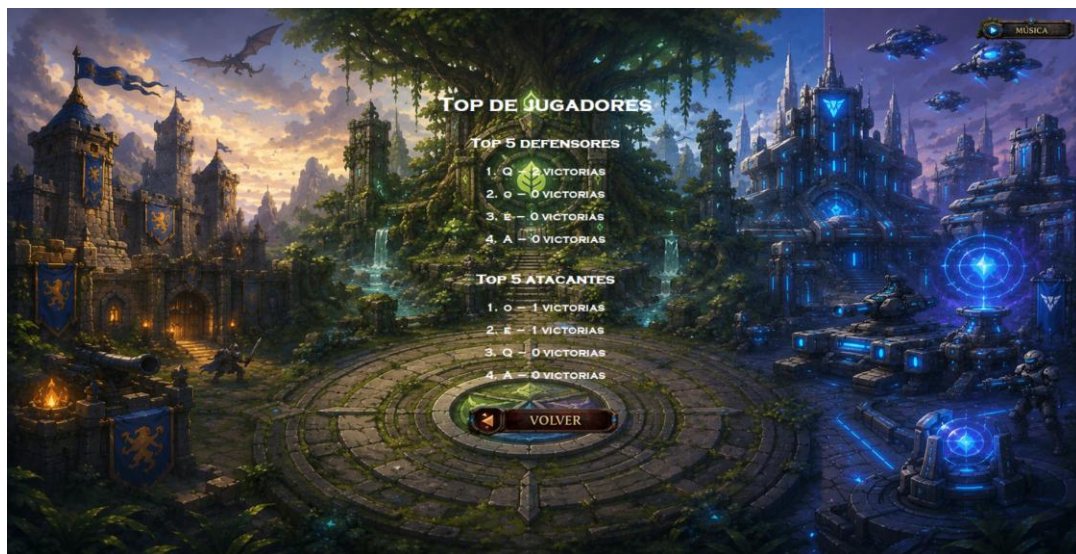
Cada facción tiene su propio conjunto de imágenes, organizadas en la carpeta `assets/facciones/<facción>/`. Para cada facción existen las imágenes de base, muro, las tres torres y las tres unidades. La única restricción de juego es que el defensor y el atacante no pueden compartir facción.

CAPTURAS DE PANTALLA

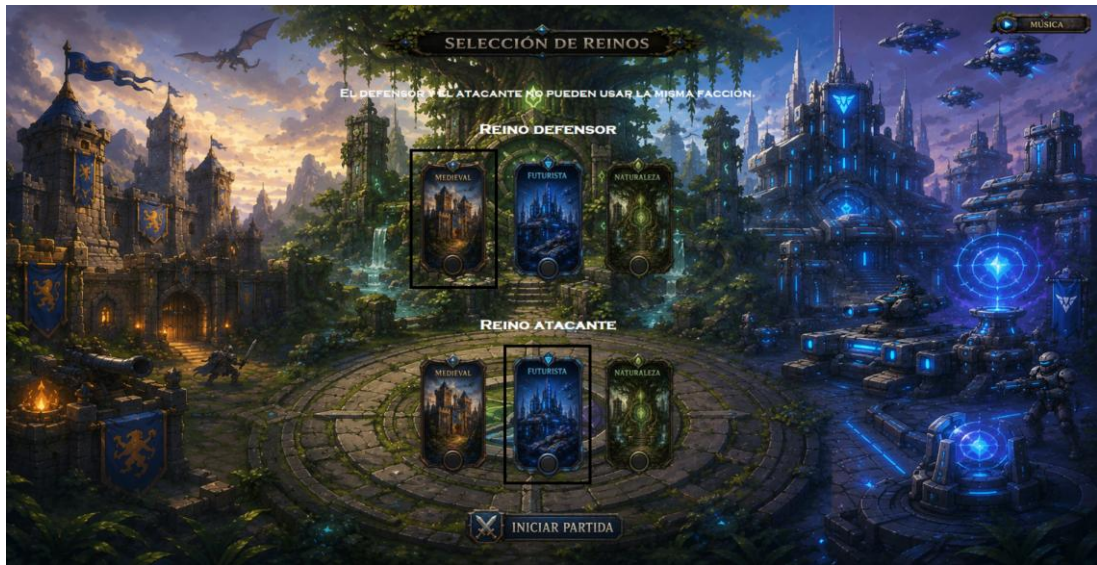
- Pantalla de inicio (registro / inicio de sesión).



- Pantalla de top de jugadores.



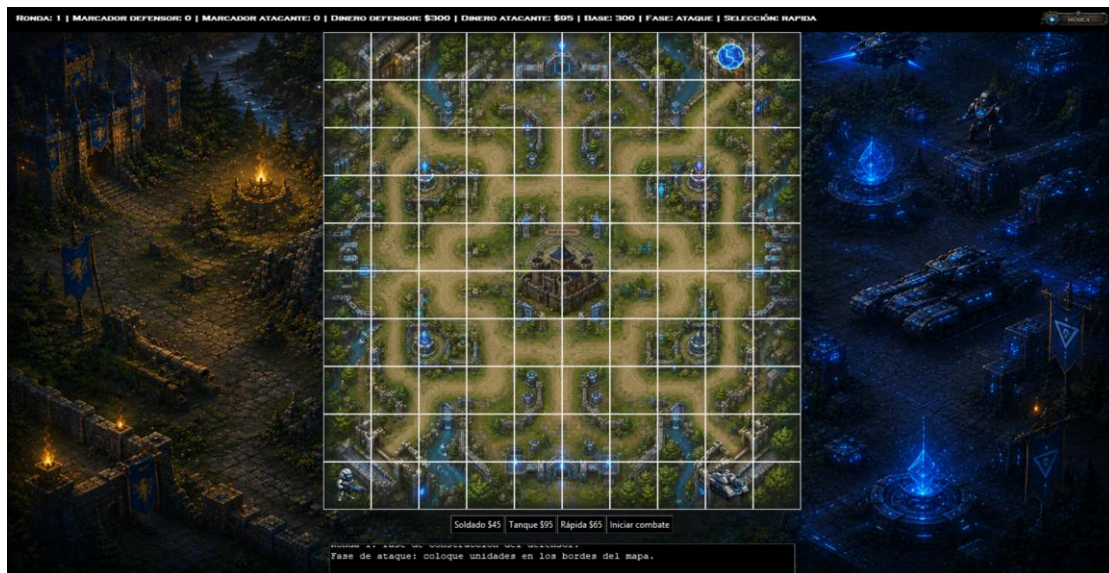
- Pantalla de selección de facciones.



- Tablero en fase de defensa (colocación de muros y torres).



- Tablero en fase de ataque (colocación de unidades).



- Tablero en fase de combate



ESTRUCTURAS DE DATOS UTILIZADAS

Programación orientada a objetos (clases)

El proyecto está organizado con cinco clases, superando el mínimo de tres exigido. Esto permite separar responsabilidades y mantener el código modular:

- Jugador: almacena el usuario, contraseña y las victorias como defensor y atacante. Incluye el método `convertir_diccionario()` para poder guardarse en JSON.
- Torre: representa una torre con su tipo, posición y estadísticas; su método `habilidad()` ejecuta el efecto especial según el tipo.
- Unidad: representa una unidad atacante con su posición, estadísticas, estado de escudo y de congelación; tiene los métodos `recibir_dano()` y `habilidad()`.
- Base Central: representa la base con su posición fija y su vida (300 puntos).
- Juego: clase principal que controla toda la lógica del juego, las fases, la economía y la interfaz gráfica.

Colecciones utilizadas

Estructura	Uso en el proyecto
Listas de objetos	self.jugadores, self.torres y self.unidades guardan, respectivamente, los jugadores cargados y los objetos en juego.
Lista de listas	self.muros guarda cada muro como [fila, columna, vida]; self.botones es la matriz 10×10 de botones del tablero.
Conjunto (set)	BASE_CELDAS guarda las cuatro tuplas (fila, columna) que ocupa la base, para comprobar colisiones de forma rápida.
Diccionarios	self.imagenes y los diccionarios de caché asocian nombres de elementos con sus imágenes; IMAGENES_SELECCION_FACCION asocia cada facción con su botón.
Constantes globales	TAMANO, BASE_FILA, BASE_COLUMNNA, TAMANO_CELDA y FACCIONES definen la configuración del tablero.
Lista de diccionarios (JSON)	Formato en el que se serializan los jugadores en el archivo jugadores.json.

EXPLICACIÓN DEL SISTEMA DE ARCHIVOS (JSON)

Toda la información de los jugadores se guarda de forma permanente en el archivo jugadores.json, ubicado junto al archivo principal. Se eligió JSON por ser un formato legible y de fácil lectura/escritura desde Python con el módulo estándar json.

El archivo contiene una lista de diccionarios. Cada diccionario representa a un jugador con cuatro claves:

- usuario: nombre de usuario.
- contraseña: contraseña del jugador.
- victorias_defensor: número de partidas ganadas como defensor.
- victorias_atacante: número de partidas ganadas como atacante.

Ejemplo del contenido del archivo:

```
[{"usuario": "o", "contraseña": "o", "victorias_defensor": 0, "victorias_atacante": 1}]
```

Lectura del archivo

El método cargar_jugadores() abre el archivo, usa json.load() para convertir su contenido en una lista de diccionarios y, a partir de cada diccionario, crea un objeto Jugador. Si el archivo no existe o falla la lectura, el juego inicia con una lista vacía, evitando que la aplicación se cierre por un error.

Escritura del archivo

El método guardar_jugadores() recorre la lista de objetos Jugador, los convierte a diccionarios con convertir_diccionario() y los escribe con json.dump() usando indent=4 (para que el archivo sea legible) y ensure_ascii=False (para conservar tildes y caracteres en español). El archivo se actualiza en dos momentos: al registrar nuevos jugadores y al finalizar una partida, cuando se suma la victoria al ganador.

INSTRUCCIONES PARA EJECUTAR EL PROGRAMA

Requisitos

- Python 3 instalado en el sistema.
- Tkinter (incluido en la instalación estándar de Python).
- Pillow, para mostrar las imágenes. Instalación: `pip install pillow`
- pygame, opcional, para la música de fondo. Instalación: `pip install pygame`

El juego está diseñado para seguir funcionando, aunque Pillow o pygame no estén instalados: en ese caso se omiten las imágenes o la música, respectivamente.

Pasos de ejecución

1. Descomprimir el archivo .zip del proyecto, manteniendo la carpeta assets junto al archivo principal.
2. Abrir una terminal en la carpeta del proyecto.
3. Ejecutar el comando: `python main.py`
4. Se abrirá la ventana del juego en la pantalla de inicio de sesión.

CONCLUSIONES INDIVIDUALES

Valeria Aguilar

El desarrollo del proyecto representó una experiencia muy enriquecedora, ya que el mismo permitió aplicar conocimientos de programación en Python utilizando Tkinter para la interfaz gráfica, Programación Orientada a Objetos para estructurar el código de manera organizada y escalable, y la biblioteca Pillow para la gestión e integración de imágenes dentro del videojuego.

En el proceso se enfrentaron diversos retos técnicos. Entre esos fue lograr una correcta organización de las clases y objetos para que cada elemento del juego, como unidades, torres, bases y jugadores, funcionara de manera independiente, pero a la vez interactuara correctamente con los demás componentes. A pesar de las dificultades, este proyecto permite desarrollar habilidades importantes de análisis, resolución de problemas y diseño de software. Enseña la importancia de planificar la estructura del programa antes de comenzar a codificar, así como la necesidad de realizar pruebas constantes para garantizar el correcto funcionamiento de cada módulo.

Ernesto Mora

El desarrollo de este proyecto permitió implementar exitosamente un juego de batalla completo en Python, con la implementación de las enseñanzas impartidas en clase, como lo son las clases POO, con las que se desarrolló gran parte del proyecto, para que se pudiera mostrar los resultados de las batallas, además se agregó música de fondo, un editor de mapas interactivo para poder seleccionar las distintas unidades ofrecidas por el juego y un registro de puntajes para poder ver los mejores cinco puestos. Lo más importante aprendido a lo largo del proyecto fue cómo estructurar un programa complejo en donde las diferentes clases se correlacionan entre sí. También se profundizó aun mas en el uso de la matriz en escenarios. Un aspecto que resultó retador fue desarrollar toda la interfaz de una forma que fuera sumamente atractiva, e integrar los recursos gráficos sin afectar el funcionamiento del programa.

BIBLIOGRAFÍA

Python Software Foundation. «tkinter — Interfaz de Python para Tcl/Tk». Documentación oficial de Python. <https://docs.python.org/es/3/library/tkinter.html>

Python Software Foundation. «json — Codificador y decodificador JSON». Documentación oficial de Python. <https://docs.python.org/es/3/library/json.html>

Python Software Foundation. «Clases» (Programación Orientada a Objetos). Tutorial oficial de Python. <https://docs.python.org/es/3/tutorial/classes.html>

Pygame Community. «pygame.mixer.music — Control de la reproducción de música». Documentación oficial de pygame. <https://www.pygame.org/docs/ref/music.html>

Pillow (PIL Fork). «ImageTk Module — Imágenes compatibles con Tkinter». Documentación oficial de Pillow. <https://pillow.readthedocs.io/en/stable/reference/ImageTk.html>

TkDocs. «Tkinter Tutorial — Canvas» (uso del lienzo y de window items para colocar widgets e imágenes). <https://tkdocs.com/tutorial/canvas.html>

GeeksforGeeks. «Playing audio file in Pygame» (carga y reproducción de música de fondo). <https://www.geeksforgeeks.org/python/python-playing-audio-file-in-pygame/>

CodersLegacy. «Pillow ImageTk (Using PIL with Tkinter)» (cargar y redimensionar imágenes en Tkinter). <https://coderslegacy.com/python/pillow-imagetk-tkinter/>

Anthropic. (2025). Claude (claude-sonnet-4-20250514) [Modelo de lenguaje]. <https://www.anthropic.com>
